

XIAMEN UNIVERSITY MALAYSIA
厦門大學馬來西亞分校

贪心算法

Greedy Algorithms: Local Optimum and Global Optimum

作者：Wu Youfan
School of Computing and Data Science, Xiamen University Malaysia

2026 年 10 月 18 日

目录

1. 贪心的本质
2. 贪心的正确性
3. 辅助工具：优先队列
4. 经典例题
5. 总结



目录

1 贪心的本质

2 贪心的正确性

3 辅助工具：优先队列

4 经典例题

5 总结

什么是贪心算法

贪心算法：在每一步选择中都采取当前状态下最好或最优（即最有利）的选择，从而希望导致结果是全局最好或最优的算法。

生活中的类比

- 贪心策略贯穿人的一生，人们总以为每一步做出局部最优解，就会变成全局最优。
- 但人们总忘了证明贪心正确性，局部最优有时会误入歧途。
- 试着用长远眼光代替短浅目光，用动态规划代替贪心算法。

核心特征

- **无后效性**：当前选择不会影响后续状态的结构，只改变数值。
- **局部最优**：只在当前步做决策，不回溯。

局部最优不一定等于全局最优

为什么不能盲目贪心

反例：硬币找零问题

问题描述：给定面值为 1、3、4 的硬币，要找零 6 元，求最少硬币数。

贪心策略：每次选不超过余额的最大面额。

- 第 1 步：选 4 元，余额剩 2 元。
- 第 2 步：选 1 元，余额剩 1 元。
- 第 3 步：选 1 元，余额剩 0 元。
- 总共用了 3 枚硬币。

真正的最优解

- 选两个 3 元硬币。
- 总共只用 2 枚硬币。

贪心策略在没有证明之前，只是一种启发式猜想

目录

1 贪心的本质

2 贪心的正确性

3 辅助工具：优先队列

4 经典例题

5 总结



贪心成立的条件

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

一个贪心算法是正确的，必须同时满足两个性质：

1. 贪心选择性质

- 全局最优解可以通过一系列局部最优的选择（即贪心选择）来达到。
- 每次选择可以依赖之前的选择，但不依赖未来的选择。

2. 最优子结构

- 问题的最优解包含其子问题的最优解。
- 做出贪心选择后，原问题转化为规模更小的子问题。

证明贪心正确性的常用方法：交换论证法

- 假设存在一个最优解，它不包含贪心选择。
- 尝试通过替换，将最优解变成包含贪心选择的解，且总代价不增加。
- 从而证明贪心选择一定存在于某个最优解中。

目录

1 贪心的本质

2 贪心的正确性

3 辅助工具：优先队列

4 经典例题

5 总结

优先队列（堆）的引入

问题：在许多贪心问题中，我们需要高频地“取出当前最大/最小值”，并“插入新元素”。

普通数组：找最值需要遍历整个数组，时间复杂度 $O(N)$。**优先队列**：找最值与插入的时间复杂度均为 $O(\log N)$。

C++ STL 提供的大根堆与小根堆

```
1 priority_queue<int> q;                                  // 优先队列 大根堆
2 priority_queue<int, vector<int>, greater<int>> qq; // 小根堆
```

常用操作

- `q.empty()`：返回 true 代表优先队列为空。
- `q.size()`：返回优先队列里有几个数。
- `q.push(x)`：把 x 加入到优先队列中。

优先队列（堆）的引入

- `q.pop()`：弹出（删除）优先队列的堆顶元素（最大/最小值）。
- `q.top()`：返回优先队列中的堆顶元素（最大/最小值）。

目录

1 贪心的本质

2 贪心的正确性

3 辅助工具：优先队列

4 经典例题

5 总结



例题：合并果子

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

题目：有 n 堆果子，每一堆有 a_i 个。每次只能挑选两堆合并，合并的代价是这两堆果子的数量之和。问将所有的果子合并成一堆，总代价最小是多少？

示例 1

- 输入：3 堆果子，数量分别为 1、2、9
- 输出：15

示例 2

- 输入：4 堆果子，数量分别为 1、2、3、4
- 输出：19

示例 3

- 输入：5 堆果子，数量分别为 5、4、3、2、1
- 输出：33

合并果子：解法

贪心策略：每一次选取当前两堆数量最少的果子合并。

为什么这样贪心是对的

- 合并产生的代价等于新堆的数量，新堆会被再次参与合并。
- 所以越早合并的堆，它的数值会被累加更多次。
- 为了让总代价最小，应该让较小的数值累加更多次，较大的数值尽量晚合并。

实现思路

- 把所有果子放入小根堆（优先队列）。
- 每次取出两个最小值 x, y，累加代价 `ans += x + y`。
- 将合并后的 $x + y$ 重新放回小根堆。
- 直到堆里只剩下一个元素。

合并果子：解法

参考代码

```
int ans = 0;
while (qq.size() > 1) {
    int x = qq.top(); qq.pop();
    int y = qq.top(); qq.pop();
    ans += x + y;
    qq.push(x + y);
}
cout << ans << endl;
```

例题：区间调度

题目：有 n 个区间，每个区间由起始时间 l_i 和结束时间 r_i 构成。请选出尽量多的区间，使得它们两两不重叠。

示例 1

- 输入：3 个区间，分别为 [1, 3], [2, 4], [3, 5]
- 输出：2（可以选择 [1, 3] 和 [3, 5]）

示例 2

- 输入：4 个区间，分别为 [1, 2], [2, 3], [3, 4], [1, 4]
- 输出：3（可以选择 [1, 2], [2, 3], [3, 4]）

示例 3

- 输入：3 个区间，分别为 [5, 8], [6, 7], [8, 9]
- 输出：2（可以选择 [6, 7] 和 [8, 9]）

区间调度：解法

错误的贪心直觉

- 选开始最早的区间：可能这个区间极其漫长，占据了所有时间。
- 选持续时间最短的区间：可能它卡在正中间，把两边的时间截断了。

正确的贪心策略

- 每次选择结束时间最早的区间。
- 为什么？结束得越早，留给后面区间的时间就越多。

算法过程

1. 按区间的结束时间 r_i 从小到大排序。
2. 贪心地选择第一个区间。
3. 依次遍历后面的区间，如果它的开始时间大于等于上一个选择的结束时间，就选择它。

排序是贪心的常客：选定一个排序标准，再依次做局部决策

目录

1 贪心的本质

2 贪心的正确性

3 辅助工具：优先队列

4 经典例题

5 总结



总结：如何设计贪心算法

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

四步法

1. **分析结构**：确定是否能将问题分解为多个步骤。
2. **寻找贪心指标**：每次到底该挑哪个？最大的？最小的？最早结束的？
3. **尝试反例**：动手构造几个小数据，看看贪心会不会误入歧途。
4. **证明正确性**：尝试用交换论证法证明该贪心策略绝对安全。

贪心的局限性

- 贪心只管当前，一旦问题具有后效性（当前选择影响后续结构），贪心大概率会失效。
- 比如：0/1 背包问题中，只贪心性价比往往无法得到最优解。
- 贪心失效时，需要请出另一件武器：动态规划。

贪心虽好，但需证明；局部最优，不等于全局最优

附录：优先队列常见错误

XIAMEN UNIVERSITY MALAYSIA 厦門大學馬來西亞分校

错误用法	后果与修复
空队列调用 `q.top()`	程序崩溃（段错误）；需加 `q.empty()` 判断
空队列调用 `q.pop()`	程序崩溃（段错误）；需加 `q.empty()` 判断
小根堆写成 `priority_queue<int>`	变成大根堆，结果全错；应写 `greater<int>`
清空堆循环写错	不能写成 `for(int i=0;i<q.size();i++)`；正确写法是 `while(q.empty()) q.pop();`!

堆非空才取顶，小根堆多打三个参数